Chapter 7. Structuring Data with Java

7.0 Introduction

Almost every application beyond "Hello, World" needs to keep track of some structured data. A simple numeric problem might work with three or four numbers only, but most applications have groups of similar data items. A GUI-based application may need to keep track of a number of dialog windows. A personal information manager, or PIM, needs to keep track of a number of, well, persons. An operating system needs to keep track of who is allowed to log in, who is currently logged in, and what those users are doing. A library needs to keep track of who has books checked out and when they're due. A network server may need to keep track of its active clients. A pattern emerges here, and it revolves around variations of what has traditionally been called *data structuring*.

There are data structures in the memory of a running program; there is structure in the data in a file on disk, and there is structure in the information stored in a database. In this chapter, we concentrate on the first aspect: in-memory data. We'll cover the second aspect in Chapter 10; the third is out of scope for this book.

If you had to think about in-memory data, you might want to compare it to a collection of index cards in a filing box or to a treasure hunt where each clue leads to the next. Or you might think of it like my desk—apparently a scattered mess, but actually (I claim) a very powerful collection filled with meaningful information. Each of these is a good analogy for a type of data structuring that Java provides. An *array* is a fixed-length linear collection of data items, like the card filing box: it can only hold so much, then it overflows. The treasure hunt is like a data structure called a *linked list*. The first release of Java had no standard linked list class, but you could write your own traditional data structure classes (and still can; there's a demo implementation of a simplified linked list [not production ready] to show how this kind of list works, in *main/src/main/java/structure/LinkList.java*). A third category is *associative* or *map* types, also known as "key/value" collections, where each key maps to a distinct value. The variety of collections is represented by Java's `Collections` classes. A document entitled "Collections Framework Overview" exists in the Java documentation; unfortunately these tutorials have not been updated since Java 8, so they should be taken with a grain

of salt. They do provide a detailed discussion of the Collections Framework as it was in those days. The framework aspects of Java collections are summarized in Recipe 7.4.

Beware of typographic issues. The word `Arrays` (in constant width font) refers to the utility class `java.util.Arrays`; but in the normal typeface, the word "arrays" is simply the plural of "array" (and will be found capitalized at the beginning of a sentence). Similarly, both `Collections` and `Collection` are types in the framework; the former is a utility class and the latter is an interface implemented by the nonassociative `Collection` classes. Also, note that `HashMap` and `HashSet` follow the rule of having a midcapital at each word boundary, whereas the older `Hashtable` does not (the *t* is not capitalized).

The java.util package has become something of a catch-all over the years. Besides the legacy date/time API covered in Recipe 6.9, several other classes from java.util are not covered in this chapter. All the classes whose names begin with `Abstract` are, in fact, abstract, and we'll discuss their nonabstract subclasses. The `StringTokenizer` class is covered in Recipe 3.1. `BitSet` is used less frequently than some of the classes discussed here and is simple enough to learn on your own. `BitSet` stores the bits very compactly in memory, but because it predates the Collection API and wasn't retrofitted, it doesn't implement the standard collection interfaces. Also not covered here are `EnumSet` and `EnumMap`, specialized for efficient storage/retrieval of enums. These are newer than `BitSet` and *do* implement the modern collection interfaces.

We start our discussion of data structuring techniques with one of the oldest structures, the array. We'll discuss the overall structure of java.util's Collections Framework, then go through a variety of structuring techniques using classes from java.util.

7.1 Using Arrays for Data Structuring

Problem

You need to keep track of a fixed amount of information and retrieve it (usually) sequentially.

Solution

Use an array.

Discussion

Arrays can be used to hold any linear collection of data. You can make an array of any primitive type or any object type. For *arrays of primitive types*, such as `int`s and `boolean`s, the data is stored in the array. For *arrays of objects*, a reference is stored in the array, so the normal rules of reference variables and casting apply. The items in an array of primitives must all be of the same type, while those in an object array should generally all be of the same type (allowing for subtypes). Note in particular that if the array is declared as `Object[]`, object references of any type can be stored in it without casting, although a valid cast is required to take an `Object` reference out and use it as its original type. I'll say a bit more on two-dimensional arrays in Recipe 7.15; otherwise, you should treat Example 7-1 as a review example.

Example 7-1. main/src/main/java/lang/Array1.java

```
public class Array1  {
 @SuppressWarnings("unused")
 public static void main(String[] argv) {
  int[] monthLen1;     // declare a reference
  monthLen1 = new int[12];    // construct it
  int[] monthLen2 = new int[12];  // short form
  // even shorter is this initializer form:
  int[] monthLen3 = {
     31, 28, 31, 30,
     31, 30, 31, 31,
     30, 31, 30, 31,
  };

  final int MAX = 10;
  LocalDate[] days = new LocalDate[MAX];
  for (int i=0; i<MAX; i++) {
   days[i] = LocalDate.of(2022, 02, i + 1);
  }

  // Two-Dimensional Arrays
  // Want a 10-by-24 array
  int[][] me = new int[10][];
  for (int i=0; i<10; i++)
   me[i] = new int[24];

  // Remember that an array has a ".length" attribute
  System.out.println(me.length);
  System.out.println(me[0].length);
```

```
  }
}
```

Arrays in Java work well. The type checking provides reasonable integrity, and array bounds are always checked by the runtime system, further contributing to reliability. Arrays are objects, but with no additional methods. An array has a (read-only) `length` attribute, distinct for each individual array; the length is not part of the type.

See Also

The only problem with arrays is: what if the array fills up and you still have data coming in? See Recipe 7.2.

7.2 Resizing an Array

Problem

The array filled up, and you got an `ArrayIndexOutOfBoundsException`.

Solution

Make the array bigger by reallocating and copying the values.

Discussion

One approach is to allocate the array at a reasonable size to begin with; but if you find yourself with more data than will fit, reallocate a new, bigger array and copy the elements into it.[1] Example 7-2 contains code that does so.

Example 7-2. main/src/main/java/structure/Array2.java

```
public class Array2  {
   public final static int INITIAL = 10,  ❶
      GROW_FACTOR = 2;                    ❷

   public static void main(String[] argv) {
      int nDates = 0;
      LocalDateTime[] dates = new LocalDateTime[INITIAL];
      StructureDemo source = new StructureDemo(21);
```

```
        LocalDateTime c;
        while ((c=source.getDate()) != null) {

            // Suboptimal: give up
            // if (nDates >= dates.length) {
            //     throw new RuntimeException(
            //         "Too Many Dates! Simplify your life!!");
            // }

            // Better: reallocate, making data structure dynamic
            if (nDates >= dates.length) {
                LocalDateTime[] tmp =
                    new LocalDateTime[dates.length * GROW_FACTOR];
                System.arraycopy(dates, 0, tmp, 0, dates.length);
                dates = tmp;    // copies the array reference
                // old array will be garbage collected soon...
            }
            dates[nDates++] = c;
        }
        System.out.println("Final array size = " + dates.length);
    }
}
```

❶ A good guess is necessary; know your data!

❷ The growth factor is arbitrary; 2 is a good value here but will continue to double exponentially. You might want to use a factor like 1.5, which would mean more allocations at the low end but less explosive growth. You need to manage this somehow!

This technique works reasonably well for simple or relatively small linear collections of data. For data with a more variable structure, you probably want to use a more dynamic approach, as in Recipe 7.5.

7.3 Simplifying Array Handling with the Arrays Class

Problem

You need to format an array, or to compare two arrays.

Solution

Somebody suggested that you use a `for` loop. In many cases, you should not do this, but use the `Arrays` class instead:

- `Arrays.toString()` to print the values as a list
- `Arrays.equals()` for simple equality
- `Arrays.compare()` for lexicographical ("high-equals-low") comparison

Discussion

The `Arrays` class has a number of utility methods for dealing with array data. It's odd that Java language arrays, although they are objects, do not have a useful `toString()` method; that is left to the static `Arrays.toString()` method, of which there are nine overloads. There are similarly nine overloads of the static `equals()` method for comparing arrays for equality. There are 28 overloads of `compare`-type methods just for comparing arrays lexicographically!

The `static` `toString()` method has nine overloads, for arrays of any primitive type or arrays of type `Object`. The elements of the array are printed as a list, in square brackets, separated by a comma and a space. Example 7-3 demonstrates this.

Example 7-3. main/src/main/java/structure/ArraysToString.java

```
record Cat(String name){
  public String toString() { return "Meower(" + name + ")";}
}
Cat[] pets = new Cat[]{new Cat("Tom"), new Cat("Slinky")};
System.out.println("Pets are " + Arrays.toString(pets));
```

If you're unfamiliar with the use of `record` instead of `class`, refer to Recipe 8.4.

When run, Example 7-3 prints:

```
Pets are [Meower(Tom), Meower(Slinky)]
```

The static `equals()` method compares two arrays of the same type:

```
boolean Arrays.equals(a1, a2);
```

Here `a1` and `a2` are arrays of any primitive type, or arrays of (compatible) `Object` types.

The `compare()` method has overloads for all common types:

```
public static int compare(X[], X[]);
public static int compare(X[] a, int aFrom, int aTo, X[] b, int bFrom, int bTo);
```

In this example, `X` is any of the eight primitive types, or an `Object` type. These all return, quoting the JDK documentation, "0 if the first and second array are equal and contain the same elements in the same order; a value less than 0 if the first array is lexicographically less than the second array; and a value greater than 0 if the first array is lexicographically greater than the second array."

For the seven numeric types there are also `compareUnsigned()` methods:

```
public static int compareUnsigned(X[], X[]);
public static int compareUnsigned(X[] a, int aFrom, int aTo, X[], int bFrom, int bTo);
```

Finally, there are methods for comparing arrays of `Object` types:

```
public static <T extends java.lang.Comparable<? super T>> int compare(T[], T[]);
public static <T extends java.lang.Comparable<? super T>> int compare(T[],
  int, int, T[], int, int);
public static <T> int compare(T[], T[], java.util.Comparator<? super T>);
public static <T> int compare(T[], int, int, T[], int, int,
  java.util.Comparator<? super T>);
```

Here the `T` is a *generic type*; see Recipe 7.5 if you're not familiar with the concept.

The `equals()`, `compare()`, and `toString()` methods are illustrated in Example 7-4.

Example 7-4. main/src/main/java/structure/ArraysEqualsCompareToString.java

```
public class ArraysEqualsCompareToString {

    void main() {
        System.out.println("Permute int value");
        int[] ia = { 1, 2, 3, 4, 5},
              ib = { 1, 2, 3, 4, 5};
```

```
        System.out.println(
            "ia = " + Arrays.toString(ia) +
            "ib = " + Arrays.toString(ib));
        System.out.println("By Arrays.equals(): " + Arrays.equals(ia, ib));
        System.out.println("compare(ia,ib) returns " + Arrays.compare(ia, ib));
        ia[4] = 6;
        System.out.println(
            "ia = " + Arrays.toString(ia) + ", " +
            "ib = " + Arrays.toString(ib));
        System.out.println("By Arrays.equals(): " + Arrays.equals(ia, ib));
        System.out.println("compare(ia,ib) returns " + Arrays.compare(ia, ib));
```

To save a tree, this shows one segment of the code; it is repeated with an array of a different type, and another with the contents of one element shuffled. The output of this segment is as follows:

```
Permute int value
ia = [1, 2, 3, 4, 5], ib = [1, 2, 3, 4, 5]
By Arrays.equals(): true
compare(ia,ib) returns 0
ia = [1, 2, 3, 4, 6], ib = [1, 2, 3, 4, 5]
By Arrays.equals(): false
compare(ia,ib) returns 1
```

7.4 The Collections Framework

Problem

You're having trouble keeping track of all these lists, sets, and iterators.

Solution

There's a pattern to it. See Table 7-1 and Figure 7-1.

Discussion

`List`, `Set`, `Map`, and `Queue` are the four fundamental data structures of the Collections Framework. `List` and `Set` are both sequences, with the difference that `List` preserves order and allows duplicate entries, whereas `Set`, true to the mathematical concept behind it, does neither, although there is one implementation of `Set` that pre-

serves order. `Map` is a key/value store, also known as a hash, a dictionary, or an associative store. `Queues` are, as the name suggests, structures that you can push into at one end and pull out from the other.

Table 7-1 shows some of the important collection-based classes from package java.util. It is far from complete, due to space limitations. Figure 7-1 shows the relationships among some of the more important types in the framework. Rectangles are interfaces; ovals, classes. Solid lines are inheritance; dashed lines represent implementations of interfaces. `Queue` and its subtypes are treated in Chapter 11.

Table 7-1. Java collections

Interfaces	**Implementations**			
	Resizable array	**Hashed table**	**Linked list**	**Balanced tree**
`List`	`ArrayList`, `Vector`		`LinkedList`	
Set		HashSet		TreeSet
`Map`		`HashMap`, `Hashtable`		`TreeMap`
Queue	Deque s, BlockingQueue s, etc.			

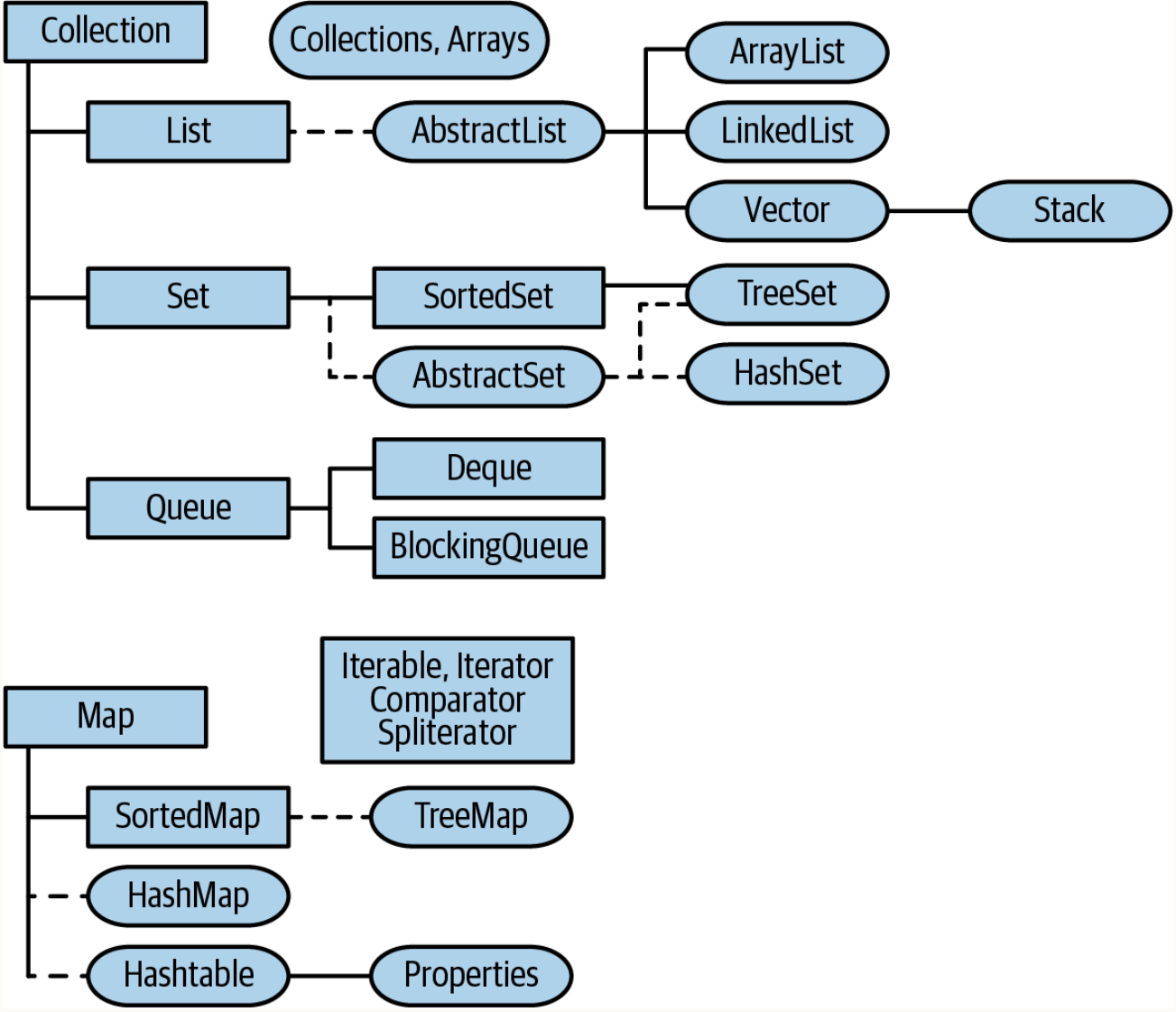


Figure 7-1. The Collections Framework

See Also

The Javadoc documentation on `Collections`, `Arrays`, `List`, `Set`, and the classes that implement them provides more details than there's room for here. Table 7-1 may further help you to absorb the regularity of the Collections Framework.

7.5 Lists: Like an Array, but More Dynamic

Problem

You don't want to worry about storage reallocation (often because you don't know how big the incoming dataset is going to be); you want a standard class to handle it for you. You want to store your data in any of the `Collections` classes defined in this chapter with type safety and without having to write downcasts when retrieving data from the collection.

Solution

Use a `List` implementation or one of the other `Collections` classes, along with Java's generic types mechanism, declaring the `Collection` with a *type parameter* identifying the type of your data. The type parameter name appears in angle brackets after the declaration, and optionally on the instantiation.

Discussion

The first of the `Collections` interface we will discuss, `List`, encapsulates the functionality of an array but allows it to expand automatically. You can just keep on adding things to it, and each addition behaves the same. Two main implementations are the `ArrayList` and the `LinkedList`.

Because the `List` implementations are classes and not part of the syntax of Java, you can't use Java's array syntax; you must use methods to access the `List`'s data. The interface has methods to add objects, retrieve objects, find objects, and tell you how big the `List` is. The `ArrayList` class has methods to say and control how big it can become without having to reallocate.

21 Java 21 adds some convenience methods: `addFirst()`, `addLast(E)`, `getFirst()`, `getLast()`, `removeFirst()`, and `removeLast()`.

Like the other `Collections` classes in java.util, `ArrayList`'s storing and retrieval methods were originally defined to have parameters and return values of `java.lang.Object`. Because `Object` is the ancestor of every defined type, you can store objects of any type in a `List` (or any collection) and cast it when retrieving it. If you need to store a small number of primitives (like `int` or `float`) in a collection, use the appropriate wrapper class (usually by auto-boxing; see the introduction to Chapter 5). To store `boolean`s, either store them directly in the specialized `java.util.BitSet` (which is neither a `List` nor a `Set` implementation; see the online documentation) or store them in a `List` using the `Boolean` wrapper class.

Because `Object` is usually too general for accurate work, Java provides the *generic types* mechanism. You declare an `ArrayList` (or other collection) with a type parameter in angle brackets, and the parameters and returns are treated as being of that type by the compiler. This ensures that objects of the wrong type don't make it into your collections, and avoids the need to write casts when retrieving objects. For example, this is how you declare an `ArrayList` for holding `String` object references:

```
List<String> myList = new ArrayList<>();
```

It is a good practice to *declare* a field as the interface type `List`, even though you are *defining* it (constructing it) as an `ArrayList`. This makes it easier to change from one `List` implementation to another, and it avoids accidentally depending on an implementation-specific method not in the `List` interface (which would also make it harder to change the implementation).

The `<>` in the definition part is a vestige of legacy Java versions, in which you had to repeat the type definition, so you'd write `new ArrayList<String>()` in that example. Nowadays just use `<>` (as in the example) to indicate that you want the type copied from the declaration. The `<>` is called the *diamond operator*.

As of Java 13, for local variables (inside a method) you can simplify by using the new `var` keyword; here, put the type parameter on the definition:

```
var myList = new ArrayList<String>();
```

Here the variable will be of type `ArrayList`, so it doesn't have the `List`-only semantics, but there is still only one place to change if you want to switch to, say, a `LinkedList`.

Table 7-2 shows some of the most important methods of the `List` interface, which is implemented by `ArrayList`, `LinkedList`, and `Vector`. This means that the same methods can be used with the older `Vector` class and any other `List`-implementing class. You'd just have to change the name used in the constructor call.

Table 7-2. Common `List<T>` methods

Method signature	Usage
`add(T o)`	Add the given element at the end
add(int i, T o)	Insert the given element at the specified position
`clear()`	Remove *all* element references from the `Collection`
contains(T o)	True if the `List` contains the given object
`forEach(lambda)`	Perform the lambda (see Chapter 9) for each element
get(int i)	Return the object reference at the specified position
`indexOf(T o)`	Return the index where the given object is found, or –1
of(T t, ...)	Create a list from multiple objects
`remove(T o), remove(int i)`	Remove an object by reference or by position
toArray()	Return an array containing the objects in the `Collection`

Example 7-5 stores data in an `ArrayList` and retrieves it for processing.

Example 7-5. main/src/main/java/structure/ArrayListDemo.java

```
public class ArrayListDemo {
  public static void main(String[] argv) {
    List<LocalDate> editions = new ArrayList<>();

    // Add lots of elements to the ArrayList...
    editions.add(LocalDate.of(2001, 06, 01));
    editions.add(LocalDate.of(2004, 06, 01));
    editions.add(LocalDate.of(2014, 06, 20));

    // Use old-style 'for' loop to get index number.
    System.out.println("Retrieving by index:");
    for (int i = 0; i<editions.size(); i++) {
      System.out.printf("Edition %d was %s\n", i + 1, editions.get(i));
    }
    // Use normal 'for' loop for simpler access.
    System.out.println("Retrieving by Iterable:");
    for (LocalDate dt : editions) {
      System.out.println("Edition " + dt);
    }
```

```
    }
}
```

The older `Vector` class predates the Collections Framework and offers additional methods with different names: `Vector` provides `addElement()` and `elementAt()`. You might still run across these in legacy code, but you should use the `List` interface methods `add()` and `get()` instead.

TIP

Another difference is that the methods of `Vector` are synchronized, so they can be accessed safely from multiple threads (see Recipe 11.4). This does mean more overhead, though, so for single-threaded access it is faster to use an `ArrayList`.

There are various conversion methods. Table 7-2 mentions `toArray()`, which will expose the contents of a `List` as an array. The `List` interface in Java 9+ features a static `of()` method, which converts in the other direction, from an array to a `List`. In conjunction with the variable arguments feature of modern Java, you can create and populate a list in one call to `List.of()`, like this:

```
List<String> firstNames = List.of("Robin", "Jaime", "Joey");
```

In legacy code that you will find in older apps and in web searches, `Arrays.asList()` provided this functionality, so you will come across code like this:

```
// Older way; no longer best
List<String> lastNames = Arrays.asList("Smith", "Jones", "MacKenzie");
// or even
List<String> lastNames =
    Arrays.asList(new String[]{"Smith", "Jones", "MacKenzie"});
```

Java does indeed get less verbose as time goes by!

Due to Java's insistence on backward compatibility—the ability to run old code on new Java versions—you can still instantiate `Collections` classes without providing a type parameter. If you do, you will get a compiler warning, and the class will behave as in the very early days; that is, the objects returned from any object-returning method will be

treated as of type `java.lang.Object`, and must be downcast before you can call any class-specific methods or use them in any application-specific method calls.

A *linked list* is commonly used when you have an unpredictably large number of data items, want to allocate just the right amount of storage, and expect to access them primarily in the order that you created them. This structure also allows for efficient insertion and removal, since the items after the change do not have to be moved, only a couple of references need to be adjusted. Each element contains both the actual data item and a reference to the "next" item, with null for the last reference at the end of the list. Figure 7-2 shows the normal arrangement.

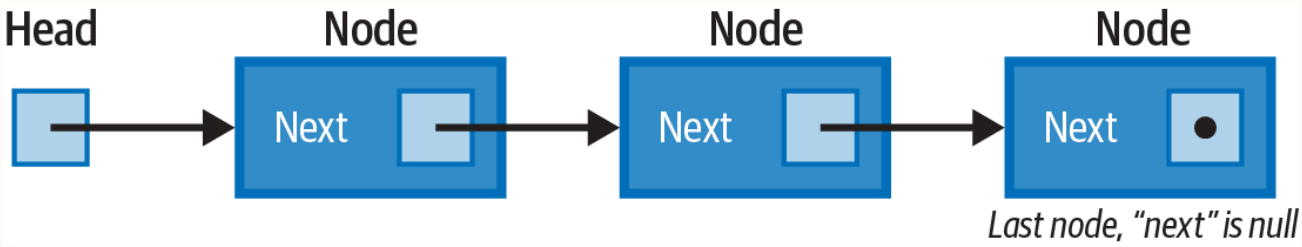


Figure 7-2. Linked list structure

The Collections API provides a `LinkedList` class; Example 7-6 is a simple program that uses it.

Example 7-6. main/src/main/java/structure/LinkedListDemo.java

```
public class LinkedListDemo {
  public static void main(String[] argv) {
    System.out.println("Here is a demo of Java's LinkedList class");
    LinkedList<String> l = new LinkedList<>();
    l.add(new Object().toString());
    l.add("Hello");
    l.add("end of the list");

    System.out.println("Here is a list of all the elements");
    l.forEach(o ->
      System.out.println("Next element: " + o));

    if (l.indexOf("Hello") != 1)
      System.err.println("Lookup does not work");
    else
      System.err.println("Lookup works");

    // Now, for added fun, let's walk the linked list backward.
    ListIterator<String> li = l.listIterator();
    // (have to get to the end first)
```

```
    while (li.hasNext())
      li.next();
    while (li.hasPrevious()) {
      System.out.println("Backwards we find: " + li.previous());
    }
  }
}
```

The `ListIterator` used here is a subinterface of `Iterator`, discussed in Recipe 7.7.

As a further example of type parameterization, consider the `Map` interface mentioned in Recipe 7.4. A `Map` requires a key and a value in its `put()` method. A `Map`, therefore, has two parameterized types. To set up a `Map` whose keys are `Person` objects and whose values are `Address` objects (assuming these two classes exist in your application), you could define it like this if it's a field:

```
Map<Person, Address> addressMap = new HashMap<>();
```

or, if it's a local variable:

```
var addressMap = new HashMap<Person, Address>();
```

This `Map` expects a `Person` as its key and an `Address` as its value in the `put()` method. The `get()` method takes one argument of type `Person` and returns an `Address` object. The `keySet()` method returns `Set<Person>` (i.e., a `Set` specialized for `Person` objects). There are also convenience routines to create a `Map` from existing objects. The most useful is several overloads of `of`, such as `Map.of(key,value,key,value...)`, similar to `List.of()` (but limited to 10 pairs), and so on.

See Also

Although the generics avoid the need to write downcasts, the casts still occur at runtime; they are just provided by the compiler. The compiler techniques used in compiling these new constructs in a backward-compatible way include *erasure* and *bridging*, topics discussed in *Java Generics and Collections* by Maurice Naftalin and Philip Wadler (O'Reilly).

7.6 Using Generic Types in Your Own Class: Stack Demo

Problem

You wish to define your own container classes using the generic type mechanism to avoid needless casting.

Solution

Define a class using `<` *TypeName* `>` where the container type is declared and *TypeName* where it is used.

Discussion

I'll demonstrate the notion of creating a generic type using a reimplementation of the `Stack` class. A criticism of the standard `Stack` class in java.util is that it extends `Vector`, such that you can remove elements from the middle of the stack, which is not "normal" for a `Stack`.

A *stack*, or more precisely a *push-down stack*, is a common data structuring operation and is often used to reverse the order of objects. The name is an analogy to a stack of papers on your desk; you add things on top and remove them from the top. A common software example is an *undo* stack in an editor, IDE, or word processor: it lets you undo edits in last-in, first-out (LIFO) order. The basic operations of a stack (in any programming language) are:

push()
: Add to stack

pop()
: Remove from stack

peek()
: Examine top element without removing

I built several versions of my own stack, culminating in the code in Example 7-7. Since the original `Stack` defines no interface of its own, I made an interface for my implementations, called `SimpleStack` to differentiate from the original. My stack has a method not in the original, `hasRoom()`. Unlike the full-blown `java.util.Stack`, `MyStack` does not expand beyond its original size, so we need a way to see if it is full without throwing an exception.

My interface and class are parameterized to take a "generic type" whose local name is `T`. This type `T` will be the type of the argument of the `push()` method, the return type of the `pop()` method, and so on. Because of this specific return type, the return value from `pop()` does not need to be downcasted. The `Container` classes in the Collections Framework (`java.util`) are parameterized similarly.

Example 7-7. main/src/main/java/structure/MyStack.java

```
public class MyStack<T> implements SimpleStack<T> {

 private int depth = 0;
 public static final int DEFAULT_INITIAL = 10;
 private T[] stack;

 public MyStack() {
  this(DEFAULT_INITIAL);
 }

 public MyStack(int howBig) {
  if (howBig <= 0) {
   throw new IllegalArgumentException(
   howBig + " must be positive, but was " + howBig);
  }
  stack = (T[])new Object[howBig];
 }

 @Override
 public boolean empty() {
  return depth == 0;
 }

 /** push - add an element onto the stack */
 @Override
 public void push(T obj) {
  // Could check capacity and expand
  stack[depth++] = obj;
 }
```

```
    /* pop - return and remove the top element */
    @Override
    public T pop() {
      --depth;
      T tmp = stack[depth];
      stack[depth] = null;
      return tmp;
    }

    /** peek - return the top element but don't remove it */
    @Override
    public T peek() {
      if (depth == 0) {
        return null;
      }
      return stack[depth-1];
    }

    public boolean hasNext() {
      return depth > 0;
    }

    public boolean hasRoom() {
      return depth < stack.length;
    }

    public int getStackDepth() {
      return depth;
    }
  }
```

The association of a particular type is done at the time the class is instantiated. For example, to instantiate a `MyStack` specialized for holding `BankAccount` objects, you would need to code only the following:

```
MyStack<BankAccount> theAccounts = new MyStack<>();
```

If you don't provide a type parameter `T`, this collection, like the ones in `java.util`, will behave as they did in the days before generic collections—accepting input arguments of any type, returning `java.lang.Object` from getter methods, and requiring downcasting—as their default, backward-compatible behavior. Example 7-8 shows a program that creates two instances of `MyStack`, one specialized for `String`s and one left general. The

general one, called `ms2`, is loaded up with the same two `String` objects as `ms1` but also includes a `Date` object. The printing code is now broken, because it will throw a `ClassCastException`: a `Date` is not a `String`. I handle this case specially for pedantic purposes: it is illustrative of the reason generic types were added to Java, and of the kinds of errors you can get into when using nonparameterized container classes.

Example 7-8. main/src/main/java/structure/MyStackDemo.java

```
public class MyStackDemo {

  @SuppressWarnings({"rawtypes","unchecked"})
  public static void main(String[] args) {
    MyStack<String> ms1 = new MyStack<>();
    ms1.push("billg");
    ms1.push("scottm");

    while (ms1.hasNext()) {
      String name = ms1.pop();
      System.out.println(name);
    }

    // Old way of using Collections: not type safe.
    // DO NOT GENERICIZE THIS
    MyStack ms2 = new MyStack();
    ms2.push("billg");            // EXPECT WARNING
    ms2.push("scottm");           // EXPECT WARNING
    ms2.push(new java.util.Date());  // EXPECT WARNING

    // Show that it is broken
    try {
      String bad = (String)ms2.pop();
      System.err.println("Didn't get expected exception, popped " + bad);
    } catch (ClassCastException ex) {
      System.out.println("Did get expected exception.");
    }

    // Removed the brokenness, print rest of it.
    while (ms2.hasNext()) {
      String name = (String)ms2.pop();
      System.out.println(name);
    }
  }
}
```

Because of this potential for error, the compiler warns that you have unchecked raw types. Like the deprecation warnings discussed in Recipe 2.7, by default, these warnings are not printed in detail by the `javac` compiler (they will appear in most IDEs). You ask for them with the rather lengthy option `-Xlint:unchecked`, but they appear automatically in JShell:

```
jshell> var list = new ArrayList();
list ==> []

jshell> list.add("Hello");
|  Warning:
|  unchecked call to add(E) as a member of the raw type java.util.ArrayList
|  list.add("Hello");
|  ^---------------^
$2 ==> true
jshell>
```

7.7 How Shall I Iterate Thee? Let Me Enumerate the Ways

Problem

You need to iterate over some structured data.

Solution

Java provides many ways to iterate over collections of data. Here they are, in newest-first order:

- `Stream.forEach()` method 8
- `IntStream.rangeClosed(start, end).forEach(Consumer c)` 8
- `Iterable.forEach()` method 8
- for-each loop 5
- `java.util.Iterator`
- `java.util.Enumeration`
- Three-part `for` loop (borrowed from 1970s C language)
- `while` loop (ditto)

Pick one and use it. Or learn them all and save!

Discussion

A few words on each of the iteration methods are given here. Note that the first few are the most common.

Stream.forEach method 8

The `Stream` mechanism introduced as part of Java's functional programming provides one of the two most recent ways of iterating, `Stream.forEach()`, and is discussed in Recipe 9.3. For now, here's a quick example, using the `BufferedReader` method `lines()` that returns a `Stream`:

```
$ jshell
jshell> import java.io.*;
jshell> BufferedReader is =
  new BufferedReader(new FileReader("/home/ian/.profile"));
is ==> java.io.BufferedReader@58651fd0
jshell> is.lines().forEach(System.out::println)
... prints the lines of the file ...
```

IntStream.rangeClosed().forEach() 8

This method allows you to iterate over a range of integer numbers. The `rangeClosed()` method is described in Recipe 5.7 and the `Consumer` interface is covered in Chapter 9.

Iterable.forEach method 8

The other recent iteration technique is the `Iterable.forEach()` method, added in Java 8. This method can be called on any `Iterable` (unfortunately, the array class does not yet implement `Iterable`) and takes one argument implementing the *functional interface* `java.util.function.Consumer`. Functional interfaces are discussed in Chapter 9, but here is one example:

```
public class IterableForEach {

    public static void main(String[] args) {
        Collection<String> c =                  ❶
                List.of("One", "Two", "Three");  ❷
        c.forEach(s -> System.out.println(s));   ❸
    }
}
```

❶ Declare a `Collection` (a `Collection` is an `Iterable`).

❷ Populate it with `List.of()` with a sequence of objects (see Recipe 7.5 for how this arbitrary argument list becomes a `List`).

❸ Invoke the collection's `forEach()` method, passing a lambda expression (see Chapter 9 for a discussion of how `s -> System.out.println(s)` gets mapped to a `Consumer` interface implementation without the need to import this interface).

This style of iteration—sometimes called *internal iteration*—inverts the control from the traditional `for` loop; the collection is in charge of when and how the iteration works.

TIP

Both `Stream.forEach` and `Iterable.forEach()` take one argument, of type `java.util.function.Consumer`, so they work largely the same way, at least syntactically. This is intentional.

Java for-each loop

This is the for-each loop syntax:

```
for (Type var : Iterable<Type>) {
      // do something with "var"
}
```

The for-each loop is probably the most common style of loop in modern Java code. The `Iterable` can be an array or anything that implements `Iterable` (the `Collection` implementations included).

This style is used throughout the book. In addition, many third-party frameworks/libraries provide their own types that implement `Iterable` for use with the `for` loop.

Three-part for loop

This is the traditional `for` loop invented by Dennis Ritchie in the early 1970s for the C language:

```
for (init; test; change) {
    // do something
}
```

Its most common form is with an `int` "index variable" or "loop variable":

```
MyDataType[] data = ...
for (int i = 0; i < data.length; i++)
    MyDataType d = data[i];
    // do something with 'd'
}
```

while loop

A `while` loop executes its loop body as long as (while) the test condition is true. It's commonly used in conjunction with an `Enumeration` or `Iterator`, like this:

```
Iterator<MyData> iterator = ...
while (iterator.hasNext()) {
    MyData md = iterator.next();
    //
}
```

java.util.Iterator

The older `Iterator` interface has three methods:

```
public interface java.util.Iterator<E> {
  public abstract boolean hasNext();
  public abstract E next();
  public default void remove();
}
```

It was once common to write code like this, which you'll still find occasionally in older code:

```
Iterator it = ...; // legacy code; might not even have type parameter
while (it.hasNext()) {
    (MyDataType) c = it.next();
```

```
        // Do something with c
    }
```

The `remove()` method in the `Iterator` throws an `UnsupportedOperationException` if called on a read-only collection. In conjunction with `Stream`s and default methods, there is now a fourth method:

```
public default void forEachRemaining(java.util.function.Consumer<? super E>);
```

Enumeration

An `Enumeration` is like an `Iterator` (shown earlier), but it lacks the `remove()` method, and the control methods have longer names—for example, `hasMoreElements()` and `nextElement()`. For new code, there is little to recommend using `Enumeration`.

7.8 Avoiding Duplicate Values with a Set

Problem

You want a structure that will avoid storing duplicates.

Solution

Use a `Set` implementation instead of a `List` (e.g., `Set<String> myNames = new HashSet<>()`).

Discussion

The `Set` interface is similar to the `List` interface,[2] with methods like `add()`, `remove()`, `contains()`, `size()`, and `isEmpty()`. The difference is that a `Set` doesn't preserve order; instead, it enforces uniqueness—if you add the same item (as considered by its `equals()` method twice or more, it will only be present once in the set. For this reason, the index-based methods such as `add(int, Object)` and `get(int)` are missing from the `Set` interface: you might know that you've added seven objects but only five of those were unique, so calling `get()` to retrieve the sixth one would throw an `ArrayIndexOutOfBoundsException`! It's better not to think of a `Set` as being indexed, because it isn't. Instead, the `contains()` method can be used to determine whether a given object is or is not currently in the `Set`.

WARNING

As the Java `Set` document states: "Note: Great care must be exercised if mutable objects are used as set elements. The behavior of a set is not specified if the value of an object is changed in a manner that affects `equals` comparisons while the object is an element in the set. A special case of this prohibition is that it is not permissible for a set to contain itself as an element." Basically, just store immutable objects in sets.

Example 7-9 shows a duplicate entry being made to a `Set`, which will contain only one copy of the string `"One"`.

Example 7-9. main/src/main/java/structure/SetDemo.java

```
Set<String> hashSet = new HashSet<>();
hashSet.add("One");
hashSet.add("Two");
hashSet.add("One"); // DUPLICATE
hashSet.add("Three");
System.out.println(hashSet);
```

Not surprisingly, only the three distinct values are printed:

```
[One, Two, Three]
```

The results are not guaranteed to come out in any particular order. But, if you change the order of the add calls, the list does come out in the same order, which is defined as the hashing order. Items are maintained in the order of their `hashCode()` values; the dedicated reader can explore this via the online version of the *SetDemo.java* file.

If you need a `Set` sorted, there is a `SortedSet` interface, of which the most common implementation is `TreeSet`; see a `TreeSet` example in "TreeSet Example". There are constructor overloads that accept a `Comparator`; the default is to use the class's natural order.

As with `List`s, the `Set` interface offers the `of` method as of Java 9:

```
Set<Double> nums = Set.of(Math.PI, 22D/7, Math.E);
Set<String> firstNames = Set.of("Robin", "Jaime", "Joey");
```

There are many other methods in the `Set` interface. Although there are no methods explicitly named for set union and intersection operations, these are easily implemented using the `addAll()` and `retainAll()` methods in the `Set` interface. An example is shown in *main/src/main/java/structure/SetArithmetic.java*, of both creating temporary `Set`s to hold the result, and using Streams `Collectors` (see Chapter 9) with no temporary objects.

7.9 Mapping with Hashtable and HashMap

Problem

You need a one-way mapping from one data item to another.

Solution

Use a `Map` implementation, such as a `HashMap`.

Discussion

`Map`s provide a one-way mapping from one set of object references to another. They are completely general purpose. I've used them to map from Swing push buttons to the URL that is to be opened when the button is pushed, to map names to addresses, and to implement a simple in-memory cache in a web server. You can map from anything to anything. You'd usually want to provide type parameters for both the keys and the values. Example 7-10 maps from company names to `Address`es; the `Address` class is a trivial `record` (see Recipe 8.4) to keep the example short.

Example 7-10. main/src/main/java/structure/HashMapDemo.java

```
public class HashMapDemo {

  record Address(String city, String state){}

  public static void main(String[] argv) {

    // Construct and load the hash. This simulates loading a
    // database or reading from a file, or wherever the data is.

    Map<String,Address> map = new HashMap<>();
```

```
    // Populate it: the map maps from company name to Address.
    map.put("Adobe", new Address("Mountain View", "CA"));
    map.put("IBM", new Address("White Plains", "NY"));
    map.put("Learning Tree", new Address("Los Angeles", "CA"));
    map.put("Microsoft", new Address("Redmond", "WA"));
    map.put("O'Reilly", new Address("Sebastopol", "CA"));
    map.put("Rejminet Group", new Address("Toronto", "ON"));

    // Several versions of the "retrieval" phase.
    // Version 1: get one pair's value given its key
    // (presumably the key would really come from user input):
    System.out.println("Version 1");
    String queryString = "O'Reilly";
    System.out.println("You asked about " + queryString + ".");
    var result = map.get(queryString);
    System.out.println("They are located in: " + result);
    System.out.println();

    // Version 2: get ALL the keys and values
    // (maybe to print a report, or to save to disk)
    System.out.println("Version 2");
    for( String key : map.keySet()) {
      System.out.println("Key " + key +
        "; Value " + map.get(key));
    }
    System.out.println();

    // Version 3: Same but using a Map.Entry lambda
    System.out.println("Version 3");
    map.entrySet().forEach(mE ->
      System.out.println("Key " + mE.getKey()+
        "; Value " +mE.getValue()));
    System.out.println();
  }
}
```

For this version we used both a `for` loop and a `forEach()` loop; the latter uses the return from `entrySet()`, a set of `Map.Entry`, each of which contains one key and one value (this may be faster on large maps because it avoids going back into the map to get the value each time through the loop). If you are modifying the map as you are going through it (e.g., removing elements), either inside the loop or in another thread, then these forms would fail with a `ConcurrentModificationException`. You then need to use an `Iterator` explicitly to control the loop:

```
// Version 2: get ALL the keys and values
// with concurrent modification
Iterator<String> it = map.keySet().iterator();
while (it.hasNext()) {
 String key = it.next();
 if (key.equals("Sun") || key.equals("Netscape")) {
  it.remove();
  continue;
 }
 System.out.println("Company " + key + "; " +
  "Address " + map.get(key));
}
```

A more functional (see Chapter 9) way of writing the removal, not involving explicit looping, would be this:

```
// Alternate 1 to just do the removals, without explicit looping
map.keySet().removeIf(key -> Set.of("Netscape", "Sun").contains(key));

// Alternate 2 to do the removals without an explicit loop
map .entrySet()
 .removeIf(entry -> Set.of("Netscape", "Sun")
  .contains(entry.getKey()));

// Show the results.
map.entrySet().forEach(System.out::println);
```

Both of the examples are from *javasrc/main/src/main/java/structure/HashMapWithRemoves.java*.

TIP

`HashMap` methods are not synchronized. The older and similar `Hashtable` methods are synchronized, for use with multiple threads, with some overhead.

7.10 Storing Strings in Properties and

Preferences

Problem

You need to store keys and values that are both strings, possibly with persistence across runs of a program—for example, program customization.

Solution

Use a `java.util.prefs.Preferences` object or a `java.util.Properties` object.

Discussion

Here are two approaches to customization based on the user's environment. Java offers `Preferences` and `Properties` for cross-platform customizations.

Preferences

The `Preferences` class `java.util.prefs.Preferences` provides an easy-to-use mechanism for storing user customizations. While the API is totally platform-neutral, the prefs will be stored in a system-dependent way (which might mean dot files on Unix, a preferences file on the Mac, or the registry on Windows systems). This class provides a hierarchical set of nodes representing a user's preferences. Data is stored in the system-dependent storage format but can also be exported to or imported from an XML format. Example 7-11 is a simple demonstration of `Preferences`.

Example 7-11. main/src/main/java/structure/PrefsDemo.java

```
public class PrefsDemo {

  public static void main(String[] args) throws Exception {

    // Set up the Preferences for this application, by class.
    Preferences prefs = Preferences.userNodeForPackage(PrefsDemo.class);

    // Retrieve some preferences previously stored, with defaults in case
    // this is the first run.
    String text    = prefs.get("textFontName", "lucida-bright");
    String display = prefs.get("displayFontName", "lucida-blackletter");
    System.out.println(text);
    System.out.println(display);
```

```
    // Assume the user chose new preference values: Store them back.
    prefs.put("textFontName", "times-roman");
    prefs.put("displayFontName", "helvetica");

    // Toss in a couple more values for the curious who want to look
    // at how Preferences values are actually stored.
    Preferences child = prefs.node("a/b");
    child.putInt("meaning", 42);
    child.putDouble("pi", Math.PI);

    // And dump the subtree from our first node on down, in XML.
    prefs.exportSubtree(System.out);
  }
}
```

When you run the `PrefsDemo` program the first time, of course, it doesn't find any settings, so the calls to `preferences.get()` return the default values:

```
$ java -cp target/classes structure.PrefsDemo
lucida-bright
lucida-blackletter
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!DOCTYPE preferences SYSTEM "http://java.sun.com/dtd/preferences.dtd">
<preferences EXTERNAL_XML_VERSION="1.0">
  <root type="user">
    <map/>
    <node name="structure">
      <map>
        <entry key="displayFontName" value="helvetica"/>
        <entry key="textFontName" value="times-roman"/>
      </map>
      <node name="a">
        <map/>
        <node name="b">
          <map>
            <entry key="meaning" value="42"/>
            <entry key="pi" value="3.141592653589793"/>
          </map>
        </node>
      </node>
    </node>
  </root>
</preferences>
```

On subsequent runs, it finds and returns the user-provided settings (I've elided the XML output from the second run because most of the XML output is the same):

```
> java structure.PrefsDemo
times-roman
helvetica
...
>
```

Properties

The `Properties` class is similar to a `HashMap` or `Hashtable` (it actually extends the latter) but with methods defined specifically for string storage and retrieval and for loading/saving. `Properties` objects are used throughout Java, for everything from setting the platform font names to customizing user applications into different `Locale` settings as part of internationalization and localization. When stored on disk, a `Properties` object looks just like a series of `name=value` assignments, with optional comments. Comments are added when you edit a `Properties` file by hand, ignored when the `Properties` object reads itself, and lost when you ask the `Properties` object to save itself to disk. Here is an example of a `Properties` file that could be used to internationalize the menus in a GUI-based program:

```
# Default properties for MenuIntl
program.title=Demonstrate I18N (MenuIntl)
program.message=Welcome to an English-localized Java Program
#
# The File Menu
#
file.label=File Menu
file.new.label=New File
file.new.key=N
file.open.label=Open...
file.open.key=O
file.save.label=Save
file.save.key=S
file.exit.label=Exit
file.exit.key=Q
```

Here is another example, showing some personalization properties:

```
name=Ian Darwin
favorite_popsicle=cherry
favorite_rock group=Fleetwood Mac
favorite_programming_language=Java
pencil_color=green
```

A `Properties` object can be loaded from a file. The rules are flexible: either `=`, :, or spaces can be used after a key name and its values. Spaces after a nonspace character are ignored in the key. A backslash can be used to continue lines or to escape other characters. Comment lines may begin with either `#` or `!`. Thus, a `Properties` file containing the previous items, if prepared by hand, could look like this:

```
# Here is a list of properties
! first, my name
name Ian Darwin
favorite_popsicle = cherry
favorite_rock\ group \
 Fleetwood Mac
favorite_programming_language=Java
pencil_color green
```

Fortunately, when a `Properties` object writes itself to a file, it uses the following simple format:

```
key=value
```

Here is an example of a program that creates a `Properties` object and adds into it the list of companies and their locations from Recipe 7.9. It then loads additional properties from disk. To simplify the I/O processing, the program assumes that the `Properties` file to be loaded is contained in the standard input, as would be done using a command-line redirection:

```
public class PropsCompanies {

  public static void main(String[] argv) throws java.io.IOException {

    Properties props = new Properties();

    // Get my data
    props.put("Adobe", "Mountain View, CA");
```

```
        props.put("IBM", "White Plains, NY");
        props.put("Learning Tree", "Los Angeles, CA");
        props.put("Microsoft", "Redmond, WA");
        props.put("Netscape", "Mountain View, CA");
        props.put("O'Reilly", "Sebastopol, CA");
        props.put("Sun", "Mountain View, CA");

        // Now load additional properties
        props.load(System.in);

        // List merged properties, using System.out
        props.list(System.out);
     }
}
```

Running it as:

```
java structure.PropsCompanies < PropsDemo.out
```

produces the following output in the file *PropsDemo.out*:

```
-- listing properties --
Sony=Japan
Sun=Mountain View, CA
IBM=White Plains, NY
Netscape=Mountain View, CA
Nippon_Kogaku=Japan
Acorn=United Kingdom
Adobe=Mountain View, CA
Ericsson=Sweden
O'Reilly & Associates=Sebastopol, CA
Learning Tree=Los Angeles, CA
```

Similarly to the `Set` examples, in the `HashMap` and `Properties` examples, the order in which the outputs appear is neither sorted nor in the order we put them in. The hashing classes and the `Properties` subclass make no claim about the order in which objects are retrieved. If you need them sorted, see Recipe 7.11.

As a convenient shortcut, my `FileProperties` class includes a constructor that takes a filename:

```
import com.darwinsys.util.FileProperties;
...
Properties p = new FileProperties("PropsDemo.out");
```

Note that constructing a `FileProperties` object causes it to be loaded, and therefore the constructor may throw a checked exception of class `IOException`.

7.11 Sorting a Collection

Problem

You put your data into a collection in random order or used a `Properties` object that doesn't preserve the order, and now you want it sorted.

Solution

Use the static method `Arrays.sort()` or `Collections.sort()`, optionally providing a `Comparator`.

Discussion

If your data is in an array, then you can sort it using the static `sort()` method of the `Arrays` utility class. If it is in a `Collection`, you can use the static `sort()` method of the `Collections` class. Here is a set of strings being sorted in place in an `Array`:

```
public class SortArray {
  public static void main(String[] unused) {
    String[] strings = {
      "painful",
      "mainly",
      "gaining",
      "raindrops"
    };
    Arrays.sort(strings);
    for (int i=0; i<strings.length; i++) {
      System.out.println(strings[i]);
    }
  }
}
```

What if the default sort order isn't what you want? Well, you can create an object that implements the `Comparator<T>` interface and pass that as the second argument to sort. Fortunately, for the most common ordering next to the default, you don't have to write your own: a public constant `String.CASE_INSENSITIVE_ORDER` can be passed as this second argument. The `String` class defines it as a `Comparator<String>` that orders `String` objects as by `compareToIgnoreCase`. But if you need something fancier, you probably need to write a `Comparator<T>`. In some cases you may be able to use the `Comparator.comparing()` method and other static methods on `Comparator` to create a custom comparator without having to create a class. Suppose that, for some strange reason, you need to sort strings using all but the first character of the string. One way to do this would be to write this `Comparator<String>`:

```
/** Comparator for comparing strings ignoring first character.
 */
public class SubstringComparator implements Comparator<String> {
  @Override
  public int compare(String s1, String s2) {
    s1 = s1.substring(1);
    s2 = s2.substring(1);
    return s1.compareTo(s2);
    // or, more concisely:
    // return s1.substring(1).compareTo(s2.substring(1));
  }
}
```

Using it is just a matter of passing it as the `Comparator` argument to the correct form of `sort()`, as shown here:

```
public class SubstringComparatorDemo {
  public static void main(String[] unused) {
    String[] strings = {
      "painful",
      "mainly",
      "gaining",
      "raindrops"
    };
    Arrays.sort(strings);
    dump(strings, "Using Default Sort");
    Arrays.sort(strings, new SubstringComparator());
    dump(strings, "Using SubstringComparator");

    System.out.println("Functional approach:");
```

```
    Arrays.stream(strings)
      .sorted(Comparator.comparing(s->s.substring(1)))
      .forEach(System.out::println);
  }

  static void dump(String[] args, String title) {
    System.out.println(title);
    for (String s : args)
      System.out.println(s);
  }
}
```

Again, a more functional (see Chapter 9) way of writing this might be the following:

```
    System.out.println("Functional approach:");
    Arrays.stream(strings)
      .sorted(Comparator.comparing(s->s.substring(1)))
      .forEach(System.out::println);
```

Here is the output of running it:

```
$ java structure.SubstrCompDemo
Using Default Sort
gaining
mainly
painful
raindrops
Using SubstringComparator
raindrops
painful
gaining
mainly
```

And this is all as it should be.

On the other hand, you may be writing a class and want to build in the comparison functionality so that you don't always have to remember to pass the `Comparator` with it. In this case, the data class can directly implement the `java.lang.Comparable` interface, as is done by many classes in the standard API. These include the `String` class; the wrapper classes `Byte`, `Character`, `Double`, `Float`, `Long`, `Short`, and `Integer`; `BigInteger` and `BigDecimal` from `java.math`; most objects in the date/time API in `java.time`; and

`java.text.CollationKey`. Arrays or `Collections` of these types can be sorted without providing a `Comparator`.

Classes that implement `Comparable` are said to have a natural ordering. The documentation strongly recommends that a class's natural ordering be consistent with its `equals()` method. It is consistent with `equals()` if and only if `e1.compareTo((Object)e2) == 0` has the same Boolean value as `e1.equals((Object)e2)` for every instance `e1` and `e2` of the given class. This means that if you implement `Comparable`, you should also implement `equals()`, and the logic of `equals()` should be consistent with the logic of the `compareTo()` method.

If you implement `equals()`, you should also implement `hashCode()` (as discussed in "hashCode() and equals()"). Here, for example, is part of the appointment class `Appt` from a hypothetical scheduling program. The class has a `LocalDate` date variable and a `LocalTime` time variable; the latter may be null (e.g., an all-day appointment or a to-do item); this complicates the `compareTo()` function a little:

```
// public class Appt implements Comparable {
  // Much code and variables omitted - see online version
  //-----------------------------------------------------------------
  //  METHODS - COMPARISON
  //-----------------------------------------------------------------
  /** compareTo method, from Comparable interface.
   * Compare this Appointment against another, for purposes of sorting.
   * <P>Only date and time, then text, participate, not repetition!
   * (Repetition has to do with recurring events, e.g.,
   * "Meeting every Tuesday at 9").
   * This method is consistent with equals().
   * @return -1 if this<a2, +1 if this>a2, else 0.
   */
  @Override
  public int compareTo(Appt a2) {
    // If dates not same, trigger on their comparison
    int dateComp = date.compareTo(a2.date);
    if (dateComp != 0)
      return dateComp;
    // Same date. If times not same, trigger on their comparison
    if (time != null && a2.time != null) {
      // Neither time is null
      int timeComp = time.compareTo(a2.time);
      if (timeComp != 0)
        return timeComp;
    } else /* At least one time is null */ {
```

```
    if (time == null && a2.time != null) {
      return -1; // All-day appts sort low to appear first
    } else if (time != null && a2.time == null)
      return +1;
      // else both have no time set, so carry on,
  }
  // Same date & time, trigger on text
  return text.compareTo(a2.text);
}

@Override
public int hashCode() {
  final int prime = 31;
  int result = 1;
  result = prime * result + ((date == null) ? 0 : date.hashCode());
  result = prime * result + ((text == null) ? 0 : text.hashCode());
  result = prime * result + ((time == null) ? 0 : time.hashCode());
  return result;
}

@Override
public boolean equals(Object o2) {
  if (this == o2)
    return true;
  if (o2.getClass() != Appt.class)
    return false;
  Appt a2 = (Appt) o2;
  if (!date.equals(a2.date))
    return false;
  if (time != null && !time.equals(a2.time))
    return false;
  return text.equals(a2.text);
}

/** Return a String representation of this Appt.
 * Output is intended for debugging, not presentation!
 */
@Override
public String toString() {
  var sb = new StringBuilder();
  sb.append(date).append(' ');
  if (time != null) {
    sb.append(time.getHour())
    .append(':')
    .append(time.getMinute())
    .append(' ');
```

```
    } else {
      sb.append("(All day)").append(' ');
    }
    sb.append(text).toString();
    return sb.toString();
  }
```

If you're still confused between `Comparable` and `Comparator`, you're probably not alone. Table 7-3 summarizes the two comparison interfaces.

Table 7-3. `Comparable` compared with `Comparator`

Interface name	Description	Method(s)
`java.lang.Comparable<T>`	Provides a natural ordering to objects. Implemented in the class whose objects are being sorted.	`int compareTo(T o);`
`java.util.Comparator<T>`	Standalone strategy object provides full control over sorting objects of another class. Pass to `sort()` method or `Collection` constructor.	`int compare(T o1, T o2); boolean equals(T c2)`

Finally, not everything that requires order requires an explicit *sort* operation. You can avoid the overhead and elapsed time of an explicit sorting operation by ensuring that the data is in the correct order at all times, though this may or may not be faster overall, depending on your data and how you choose to keep it sorted. You can keep it sorted either manually or by using a `TreeSet` or a `TreeMap`. First, here is some code from a call tracking program that I first wrote on the very first public release of Java (the code has been modernized slightly!) to keep track of people I had had contact with and, in particular, those I needed to call back! Far less functional than a Rolodex, my `CallTrack` program maintained a list of people sorted by last name and first name. It also had the city, phone number, and email address of each person. Here is a very small portion of the code showing the "add person" operation:

```
  /** Add one (new) Person to the list, keeping the list sorted. */
  protected void add(Person p) {
    String lastName = p.getLastName();
    int i;
    // Find in "i" the position in the list where to insert this person
    for (i=0; i<usrList.size(); i++)
      if (lastName.compareTo((usrList.get(i)).getLastName()) <= 0)
```

```
        break; // If we don't break, OK, will insert at end of list.
    usrList.add(i, p);
```

WARNING

This code uses a linear search, which was fine for the original application but could get very slow on large lists (it is *O(n)*). You'd need to use hashing or a binary search to find where to put the values on large lists.

If I were writing this code today, I would likely use a `TreeSet` (which keeps objects in order) or a `TreeMap` (which keeps the keys in order and maps from keys to values; the keys would be the name and the values would be the `Person` objects). Both insert the objects into a tree in the correct order (using the same logic as an explicit sort), so an `Iterator` that traverses the tree always returns the objects in sorted order. In addition, they have methods such as `headSet()` and `headMap()`, which give a new `Set` or `Map` of objects of the same class, containing the objects lexically before a given value. The `tailSet()` and `tailMap()` methods, similarly, return objects greater than a given value, and `subSet()` and `subMap()` return a range. The `first()` and `last()` methods retrieve the obvious components from the collection. The following program uses a `TreeSet` to sort some names:

```
    // A TreeSet keeps objects in sorted order. Use a Comparator
    // published by String for case-insensitive sorting order.
    TreeSet<String> theSet = new TreeSet<>(String.CASE_INSENSITIVE_ORDER);
    theSet.add("Gosling");
    theSet.add("da Vinci");
    theSet.add("van Gogh");
    theSet.add("Java To Go");
    theSet.add("Vanguard");
    theSet.add("Darwin");
    theSet.add("Darwin");  // TreeSet is Set, ignores duplicates.

    System.out.printf("Our set contains %d elements", theSet.size());

    // Since it is sorted we can easily get various subsets
    System.out.println("Lowest (alphabetically) is " + theSet.first());

    // Print how many elements are greater than "k"
    // Should be 2 - "van Gogh" and "Vanguard"
    System.out.println(theSet.tailSet("k").toArray().length +
      " elements higher than \"k\"");
```

```
// Print the whole list in sorted order
System.out.println("Sorted list:");
theSet.forEach(name -> System.out.println(name));
```

One last point to note is that if you have a `Hashtable` or `HashMap`, you can convert it to a `TreeMap`, and therefore get it sorted, just by passing it to the `TreeMap` constructor:

```
TreeMap sorted = new TreeMap(unsortedHashMap);
```

7.12 Finding an Object in a Collection

Problem

You need to see whether a given collection contains a particular value.

Solution

Ask the collection if it contains an object of the given value.

Discussion

If you have created the contents of a collection, you probably know what is in it and what is not. But if the collection is prepared by another part of a large application, or even if you've just been putting objects into it and now need to find out if a given value was found, this recipe's for you. There are various methods available, depending on which `Collection` class you have. The methods in Table 7-4 can be used.

Table 7-4. Finding objects in a collection

Method(s)	Meaning	From interface	Implementing classes
`binarySearch()`	Fairly fast search	`Collection`	`Arrays`, `Collections`
`contains()`	Search	`Collection`	`ArrayList`, `HashSet`, `Hashtable`, `LinkList`, `Properties`, `Vector`
`containsKey()`, `containsValue()`	Checks if the collection contains the object as a `Key` or as a `Value`	`Map`	`HashMap`, `Hashtable`, `Properties`, `TreeMap`
`indexOf()`	Returns location where object is found	`Collection`	`ArrayList`, `LinkedList`, `List`, `Stack`, `Vector`
`search()`	Search	N/A	`Stack`

The methods whose names start with `contains` will (probably) use a linear search if the collection is a `Collection` (`List`, `Set`) but will be faster if the collection is hashed (`HashSet`, `HashMap`).

The next example plays a game of "find the hidden number" (or "needle in a haystack"): the player only gets one guess to see if their number is in the haystack; this trivial version always guesses the number 1,999. If the player loses, the game must report the nearest number to the player's guess, so we couldn't just use a `List` and call `List.contains()`. Instead, the numbers to look through are stored in an array. As games go, it's fairly pathetic: the computer plays against itself, so you probably know who's going to win. I wrote it that way so I would know that the data array contains valid numbers.

The interesting part is not the generation of the random numbers (discussed in Recipe 5.9). The array to be used with `Arrays.binarySearch()` must be in sorted order, but since we just filled it with random numbers, it isn't initially sorted. Hence, we call `Arrays.sort()` on the array. Then we are in a position to call `Arrays.binarySearch()`, passing in the array and the value to look for. If you run the program with a number, it runs that many games and reports on how it fared overall. If you don't bother, it plays only one game:

```
public class ArrayHunt {
  /** the maximum (and actual) number of random ints to allocate */
  protected final static int MAX    = 4000;
```

```
/** the value to look for */
protected final static int NEEDLE = 1999;
int[] haystack;
Random r;

public static void main(String[] argv) {
 ArrayHunt h = new ArrayHunt();
 if (argv.length == 0)
  h.play();
 else {
  int won = 0;
  int games = Integer.parseInt(argv[0]);
  for (int i=0; i<games; i++)
   if (h.play())
    ++won;
  System.out.println("Computer won " + won +
   " out of " + games + ".");
 }
}

/** Construct the hunting ground */
public ArrayHunt() {
 haystack = new int[MAX];
 r = new Random();
}

/** Play one game. */
public boolean play() {
 int i;

 // Fill the array with random data (hay?)
 for (i=0; i<MAX; i++) {
  haystack[i] = (int)(r.nextFloat() * MAX);
 }

 // Precondition for binary search is that data be sorted!
 Arrays.sort(haystack);

 // Look for needle in haystack
 i = Arrays.binarySearch(haystack, NEEDLE);

 if (i >= 0) {    // Found it, we win.
  System.out.println("Value " + NEEDLE +
   " occurs at haystack[" + i + "]");
  return true;
 } else {    // Not found, we lose.
```

```
      System.out.println("Value " + NEEDLE +
        " does not occur in haystack; nearest value is " +
        haystack[-(i+2)] + " (found at " + -(i+2) + ")");
      return false;
    }
  }
}
```

`Collections.binarySearch()` works almost exactly the same way, except it looks in a `Collection`, which must be sorted (presumably using `Collections.sort`, as discussed in Recipe 7.11).

7.13 Converting Between Collections and Arrays

Problem

You have a `Collection` but you need a Java language array.

Solution

Use the `Collection` method `toArray()`.

Discussion

If you have an `ArrayList` or other `Collection` and you need an array (either to deal with some other API, or just for speed), you can get it just by calling the `Collection`'s `toArray()` method. With no arguments, you get an array whose type is `Object[]`. You can optionally provide an array argument, which is used for two purposes:

- The type of the array argument determines the type of array returned.
- If the array is big enough (and you can ensure that it is by allocating the array based on the `Collection`'s `size()` method), then this array is filled and returned. If the array is not big enough, a new array is allocated instead. If you provide an array and objects in the `Collection` cannot be cast to this type, then you will get an `ArrayStoreException`.

Example 7-12 shows code for converting an `ArrayList` to an array of type `Object`.

Example 7-12. main/src/main/java/structure/ToArray.java

```
List<String> list = new ArrayList<>();
list.add("Blobbo");
list.add("Cracked");
list.add("Dumbo");

// Convert a collection to Object[], which can store objects
// of any type.
Object[] ol = list.toArray();
System.out.println("Array of Object has length " + ol.length);

String[] sl = (String[]) list.toArray(new String[0]);
System.out.println("Array of String has length " + sl.length);
```

7.14 Making Your Own Data Structures Iterable

Problem

You have written your own data structure, and you want to publish the data to be iterable so it can be used in the for-each loop.

Solution

Make your data class `Iterable`: this interace has only one method, `iterator()`, which will return an instance of your own `Iterator`. Just implement (or provide an inner class that implements) the `Iterator` interface.

Discussion

To be usable as the container object in the modern Java for-each loop, your data class must implement `Iterable`, a simple interface with one method, `Iterator<T>` `iterator()`. Whether you use this interface or want to use the older `Iterator` interface directly, the way to make data from one part of your program available in a storage-independent way to other parts of the code is to generate an `Iterator`. Here is a short program that constructs, upon request, an `Iterator` for some data that it is storing—in this case, in an array. The `Iterator` interface has only three methods—`hasNext()`, `next()`, and `remove()`—demonstrated in Example 7-13.

Example 7-13. main/src/main/java/structure/IterableDemo

```
public class IterableDemo {

 /** Demo implements Iterable, meaning it must provide an Iterator,
  * and that it can be used in a foreach loop.
  */
 static class Demo implements Iterable<String> {

  // Simple demo: use array instead of inventing new data structure
  String[] data = { "One", "Two", "Three"};

  /** This is the Iterator that makes it all happen */
  class DemoIterator implements Iterator<String> {
   int i = 0;

   /**
    * Tell if there are any more elements.
    * @return true if next() will succeed, false otherwise
    */
   public boolean hasNext() {
    return i < data.length;
   }

   /** @return the next element from the data */
   public String next() {
    return data[i++];
   }

   /** Remove the object that next() just returned.
    * An Iterator is not required to support this method, and we don't.
    * @throws UnsupportedOperationException unconditionally
    */
   public void remove() {
    throw new UnsupportedOperationException("remove");
   }
  }

  /** Method by which the Demo class makes its iterator available */
  public Iterator<String> iterator() {
   return new DemoIterator();
  }
 }

 public static void main(String[] args) {
  Demo demo = new Demo();
```

```
    for (String s : demo) {
      System.out.println(s);
    }
  }
}
```

The comments on the `remove()` method remind me of an interesting point. This interface introduces `java.util`'s attempt at something Java doesn't really have, the *optional method*. Because there is no syntax for this, and they didn't want to introduce any new syntax, the developers of the Collections Framework decided on an implementation using existing syntax. Optional methods that are not implemented will usually throw an `UnsupportedOperationException` if they ever get called. My `remove()` method does just that. `UnsupportedOperationException` is subclassed from `RuntimeException`, so it is not required to be declared or caught.

This code is simplistic, but it does show the syntax and demonstrates how the `Iterator` interface works. In real code, the `Iterator` and the data are usually separate objects (the `Iterator` might be an inner class from the data store class, as shown in Example 7-13). I show this simple example of the internals of an `Iterator` so that readers understand both how it works and how one could be provided for a more sophisticated data structure, should the need arise.

The `Iterable` interface has only one nondefault method, `iterator()`, which must provide an `Iterator` for objects of the given type. Because the `ArrayIterator` class implements this as well, we can use an object of type `ArrayIterator` in a for-each loop, as in Example 7-14.

Example 7-14. ***main/src/main/java/structure/ArrayIteratorDemo.java***

```
package structure;

import com.darwinsys.util.ArrayIterator;

public class ArrayIteratorDemo {

  private final static String[] names = {
    "rose", "petunia", "tulip"
  };

  public static void main(String[] args) {
    ArrayIterator<String> arrayIterator = new ArrayIterator<>(names);
```

```
    System.out.println("Java 5, 6 way");
    for (String s : arrayIterator) {
      System.out.println(s);
    }

    System.out.println("Java 8 ways");
    arrayIterator.forEach(s->System.out.println(s));
    arrayIterator.forEach(System.out::println);
  }
}
```

Iterable.forEach() 8

Java 8 added a `forEach` method to the `Iterable` interface, a *default method* (discussed in the introdcution to Chapter 9) that you don't have to write. Thus, without changing the `ArrayIterator`, after moving to Java 8 we can use the newest-style loop, `Iterator.forEach(Consumer)`, with a lambda expression (see Chapter 9) to print each element (see Example 7-14).

7.15 Multidimensional Structures

Problem

You need a multidimensional array or `List`. This is a common requirement in mathematical and data science applications, among others.

Solution

No problem. Java supports this.

Discussion

As mentioned in Recipe 7.1, Java arrays can hold any reference type. Since an array is a reference type, it follows that you can have arrays of arrays, or *multidimensional* arrays. And since each array has its own length attribute, the columns of a two-dimensional array, for example, do not all have to be the same length (see Figure 7-3).

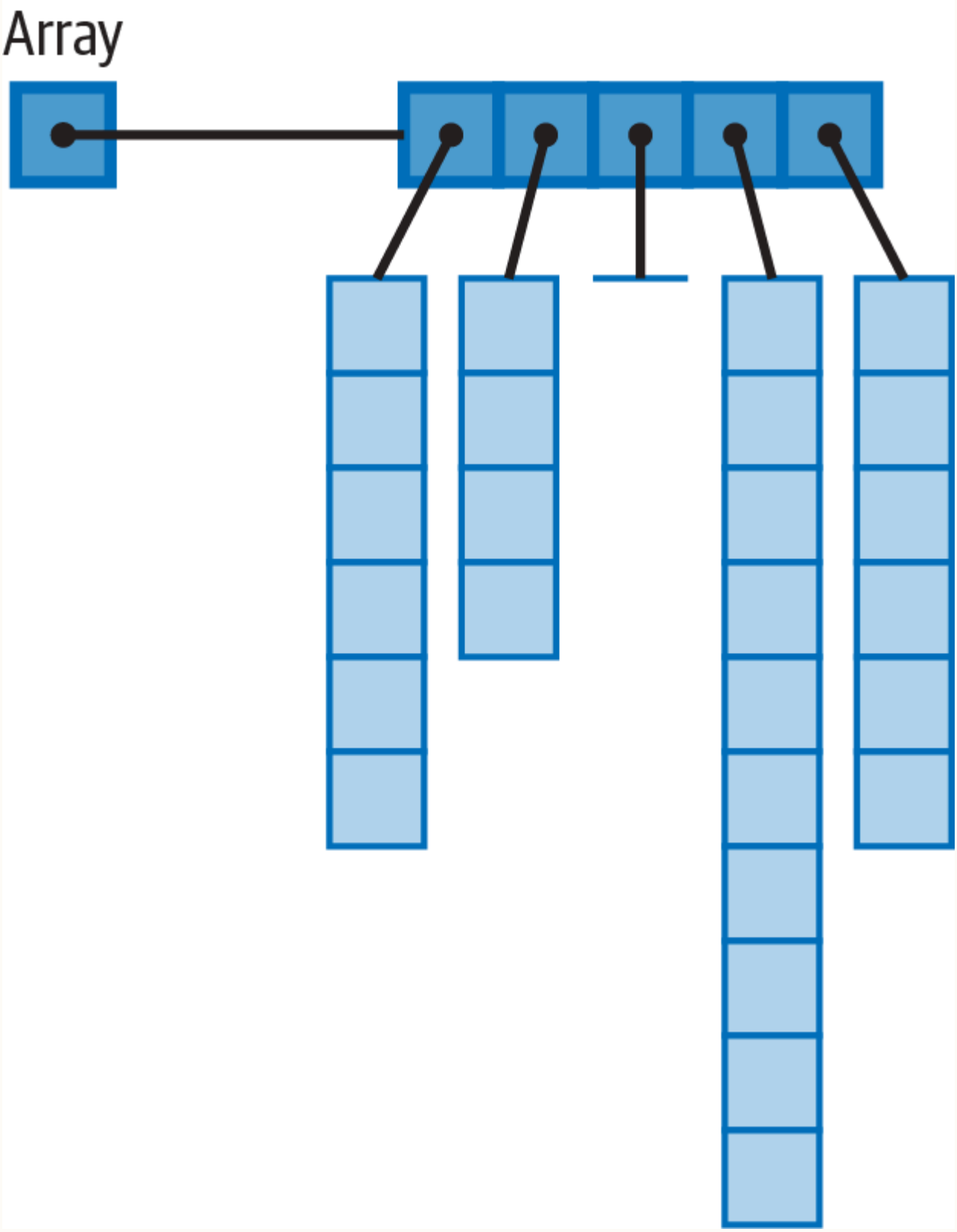


Figure 7-3. Multidimensional arrays

Here is code to allocate a couple of two-dimensional arrays, one using a loop and the other using an initializer. Both are selectively printed:

```
public class ArrayTwoDObjects {

  /** Return list of subscript names (unrealistic; just for demo). */
  public static String[][] computedArray() {
    String info[][];
    info = new String[10][10];
```

```
    for (int i=0; i < info.length; i++) {
      for (int j = 0; j < info[i].length; j++) {
        info[i][j] = "String[" + i + "," + j + "]";
      }
    }
    return info;
  }

  static String[][] inittedArray = {
    { "One", "Two" },
    { "Buckle my shoe" },
  };

  /** Run the initialization method and print part of the results */
  public static void main(String[] args) {
    print("from computedArray", computedArray());
    print("from initted array", inittedArray);
  }

  /** Print selected elements from the 2D array */
  public static void print(String tag, String[][] array) {
    System.out.println("Array " + tag + " is of length " + array.length);
    for (int i=0; i < 2; i++) {
      System.out.println("Row " + i + " is " + array[i].length);
      for (int j = 0; j < Math.min(array[i].length, 3); j++) {
        System.out.printf("Array[%d][%d] = array[%d][%d]}", i, j, i, j);
      }
    }
  }
}
```

Running it produces this output:

```
> java structure.ArrayTwoDObjects
Array from computedArray is of length 10
Row 0 is 10
Array[0][0] = String[0,0]
Array[0][1] = String[0,1]
Array[0][2] = String[0,2]
Row 1 is 10
Array[1][0] = String[1,0]
Array[1][1] = String[1,1]
Array[1][2] = String[1,2]
Array from initted array is of length 2
Row 0 is 2
```

```
Array[0][0] = One
Array[0][1] = Two
Row 1 is 1
Array[1][0] = Buckle my shoe
```

The same kind of logic can be applied to any of the `Collections`. You could have an `ArrayList` of `ArrayLists`, or a `Vector` of linked lists, or whatever your little heart desires.

As Figure 7-3 shows, it is not necessary for the array to be regular (i.e., it's possible for each column of the 2D array to have a different height). This is different from the conventional view of a matrix, which is generally assumed to be rectangular. That is why I used `array[0].length` for the length of the first column in the code example.

1 You could copy it yourself using a `for` loop if you wish, but `System.arrayCopy()` is likely to be faster because it's implemented in native code.

2 Both `List` and `Set` are subtypes of `Collection`.

Previous chapter
< 6. Dates and Times

Next chapter
8. Object-Oriented Techniques >